

Name Anisha Srivastava

Subject OS Lab

Standard

Section A

Roll No. 18F24C054 School / College

S. No.	Date	Title	Page No.	Teacher's Sign
1	24/2/26	C program to find second smallest element in an array	RB	
2	24/2/26	C program to find sum of left diagonals of a matrix		
3	3/3/26	Program to simulate cpe scheduling algorithm		
		a) FCFS b) SJF		
4	10/3/26	Program to simulate cps scheduling to find turnaround time & waiting time		
		Priority (Pre-emptive & Non Pre-emptive)		
5	17/3/26	Round Robin		
6	24/3/26	Multilevel queue scheduling		
7	31/3/26	Rate Monotonic and Earliest deadline first		
8	28/4/26	Bankers Safety Algorithm		
9	5/5/26	Bankers Algorithm and Deadlock Detection		
10	12/5/26	Memory Allocation (First Fit, Best Fit, worst)		

24/8/20

Lab-1

URBAN
EDGE
COLORFUL YEAR

1) Write a program in C to find the second smallest element in an array.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, i;
```

```
printf("Enter number of elements in the array: ");
```

```
scanf("%d", &n);
```

```
if (n < 2)
```

```
{ printf("Array must have at least 2 elements.\n");
```

```
return 0;
```

```
}
```

```
int arr[n];
```

```
printf("Enter %d elements: \n", n);
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
scanf("%d", &arr[i]);
```

```
int smallest, secondSmallest;
```

```
if (arr[0] < arr[1])
```

```
{ smallest = arr[0];
```

```
secondSmallest = arr[1];
```

```
} else
```

```
{ smallest = arr[1];
```

```
secondSmallest = arr[0];
```

```
}
```

```
for (i = 2; i < n; i++)
```

```
{ if (arr[i] < smallest)
```

```
{ secondSmallest = smallest;
```

```
smallest = arr[i];
```

```
} else if (arr[i] < secondSmallest && arr[i] != smallest)
```

```
{ secondSmallest = arr[i];
```

```
}
```

```
}  
if (smallest == secondsmallest)  
{  
    printf("There is no second smallest element.\n");  
}  
else  
{  
    printf("The second smallest element is %d\n", secondsmallest);  
}  
return 0;  
}
```

O/P

Enter the number of elements in the array: 5

Enter 5 elements:

12 7 9 5 15

The second smallest element is: 7

2) C Program to find sum of left diagonal of a matrix

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, sum=0;
```

```
    printf("Enter size of the square matrix:");
```

```
    scanf("%d", &n);
```

```
    int matrix[n][n];
```

```
    printf("Enter the elements of the matrix:\n");
```

```
    for(i=0; i<n; i++)
```

```
    { for(j=0; j<n; j++)
```

```
    {
```

```
        scanf("%d", &matrix[i][j]);
```

```
    }
```

```
    }
```

```
    for(i=0; i<n; i++)
```

```
    { sum += matrix[i][i];
```

```
    }
```

```
    printf("Sum of the left diagonal elements is: %d\n", sum);
```

```
    return 0;
```

```
}
```

DIP

Enter the size of the square matrix: 3

Enter the elements of the matrix:

1 2 3

4 5 6

7 8 9

Sum of the left diagonal elements is : 15

8/8/26

Lab-2

write a program to simulate cpu scheduling algorithm

1) FCFS and 2) SJF

1) #include <stdio.h>

int main()

{

int n, i, j;

printf("Enter number of processes:");

scanf("%d", &n);

int at[n], bt[n], ct[n], tat[n], wt[n], temp;

printf("Enter Arrival Time and Burst Time: \n");

for(i=0; i<n; i++)

{ printf("P%d AT BT: ", i+1);

scanf("%d %d", &at[i], &bt[i]);

}

for(i=0; i<n-1; i++)

{ for(j=i+1; j<n; j++)

{ if(at[i] > at[j])

{

temp = at[i];

at[i] = at[j];

at[j] = temp;

temp = bt[i];

bt[i] = bt[j];

bt[j] = temp;

}

}

}

```
ct[0] = at[0] + bt[0];
```

```
for(i=1; i < n; i++)
```

```
{ if(ct[i-1] > at[i])
```

```
    ct[i] = ct[i-1] + bt[i];
```

```
    else
```

```
        ct[i] = at[i] + bt[i];
```

```
}
```

```
for(i=0; i < n; i++)
```

```
{ lat[i] = ct[i] - at[i];
```

```
    wlat[i] = lat[i] - bt[i];
```

```
}
```

```
// Display Results
```

```
printf("\n Process | AT | BT | CT | TAT | WT | n");
```

```
for(i=0; i < n; i++)
```

```
{
```

```
    printf("P: %d | %d | %d | %d | %d | %d | %d\n", i+1, at[i], bt[i], ct[i],  
           lat[i], wlat[i],
```

```
};
```

```
return 0;
```

```
}
```

O/P

Enter number of processes: 4

Enter Arrival Time and Burst Time:

P1 AT BT: 0 3

P2 AT BT: 0 5

P3 AT BT: 0 8

P4 AT BT: 0 2

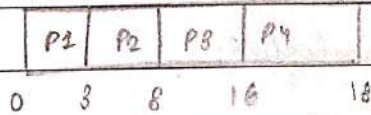
Process	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	0	5	8	8	3
P3	0	8	16	16	8
P4	0	2	18	18	16
			<u>45</u>	<u>45</u>	<u>27</u>

Avg CT = $45/4 = 11.25$

Avg TAT = $45/4 = 11.25$

Avg WT = $27/4 = 6.75$

Gantt chart



2) #include <stdio.h>

int main()

{ int n, i, choice;

printf("Enter number of processes:");

scanf("%d", &n);

int at[n], bt[n], at[n], ct[n], tat[n], ut[n], visited[n];

printf("Enter arrival time and burst time: \n");

for(i=0; i<n; i++)

{ printf("P %d AT BT: ", i+1);

scanf("%d %d", &at[i], &bt[i]);

at[i] = bt[i];

visited[i] = 0;

}

printf("\n Choose scheduling Type: \n");

printf("1. Non-Preemptive SJF \n");

printf("2. Preemptive SJF (SRTF) \n");

scanf("%d", &choice);

int completed = 0, current_time = 0, shortest;

float total_tat = 0, total_ut = 0;

if (choice == 1)

{ while (completed < n)

{ shortest = -1;

for(i=0; i<n; i++)

{ if (at[i] <= current_time & visited[i] == 0)

{

if (shortest == -1 || bt[i] < bt[shortest])

shortest = i;

}

}

if (shortest == -1)

{ current_time ++;

}

else {

current_time += bt[shortest];

ct[shortest] = current_time;

visited[shortest] = 1;

completed++;

}

else if (choice == 2)

{ while (completed < n)

{ shortest = -1;

for (i = 0; i < n; i++)

{ if (at[i] <= current_time && st[i] > 0)

{ if (shortest == -1 || st[i] < st[shortest])

shortest = i;

}

if (shortest == -1)

{ current_time++;

} else

{ st[shortest]--;

current_time++;

if (st[shortest] == 0)

{ ct[shortest] = current_time;

completed++;

}

}

for (i = 0; i < n; i++)

{ lat[i] = ct[i] - at[i];

cut[i] = lat[i] - bt[i];

total_lat += lat[i];

total_cut += cut[i];

}

```
printf("\n Process | AT | BT | CT | TAT | WT | n");
for (i = 0; i < n; i++)
{
    printf("P%d | %d | %d | %d | %d | %d | n",
           i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\n Average Turnaround Time = %.2f", total_tat/n);
printf("\n Average Waiting Time = %.2f | n", total_wt/n);
return 0;
```

Y

D/P

Enter number of processes: 4

Enter Arrival Time and Burst Time:

P1 AT BT : 0 3

P2 AT BT : 0 5

P3 AT BT : 0 8

P4 AT BT : 0 2

Choose Scheduling Type:

1. Non-Preemptive SJF

2. Preemptive SJF (SRTF)

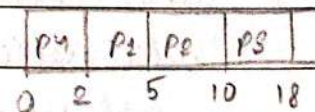
Enter choice: 2

Process	AT	BT	CT	TAT	WT
P1	0	3	5	5	2
P2	0	5	10	10	5
P3	0	8	18	18	10
P4	0	2	2	2	0

Gantt chart

Average Turnaround Time = 8.25

Average waiting Time = 4.25



comparison

FCFS

Avg TAT = 11.25

Avg WT = 6.25

SJF (Preemptive) SRTF

Avg TAT = 8.25

Avg WT = 4.25

SJF is better than FCFS. It takes less time

10/3/26

Lab 3

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
Priority (pre-emptive and non-pre-emptive)

```
#include <stdio.h>
```

```
void nonPreemptivePriority(int n, int at[], int bt[], int pr[])
```

```
{
```

```
    int ct[20], tat[20], wt[20], finished[20] = {0};
```

```
    int time = 0, completed = 0;
```

```
    while (completed < n)
```

```
    {
```

```
        int idx = -1, minPr = 9999;
```

```
        for (int i = 0; i < n; i++)
```

```
        { if (!finished[i] && at[i] <= time && pr[i] < minPr)
```

```
            { minPr = pr[i];
```

```
              idx = i;
```

```
            }
```

```
        }
```

```
        if (idx == -1)
```

```
        { time++;
```

```
          continue;
```

```
        }
```

```
        time += bt[idx];
```

```
        ct[idx] = time;
```

```
        tat[idx] = ct[idx] - at[idx];
```

```
        wt[idx] = tat[idx] - bt[idx];
```

```
        finished[idx] = 1;
```

```
        completed++;
```

```
    }
```

```
    timeline[len] = time;
```

```
    process[len] = idx + 1;
```

```
    len++;
```

```

printf("\n -- Non-Preemptive Priority Scheduling -- \n");
printf("PID | AT | BT | PR | CT | TT | WT | \n");
for (int i = 0; i < n; i++)
    printf("%d | %d | %d | %d | %d | %d | %d | \n", i+1, at[i], bt[i],
        pr[i], ct[i], tt[i], wt[i]);
}
printGanttChart(timeline, process, len);
}

void preemptivePriority (int n, int at[], int bt[], int pr[])
{
    int ct[20], tat[20], wt[20], rt[20], finished[20] = {0};
    int time = 0, completed = 0;
    for (int i = 0; i < n; i++)
        rt[i] = bt[i];
    while (completed < n)
    {
        int idx = -1, minPr = 9999;
        for (int i = 0; i < n; i++)
        {
            if (!finished[i] && rt[i] <= time && pr[i] < minPr)
            {
                minPr = pr[i];
                idx = i;
            }
        }
        if (idx == -1)
        {
            time++;
            continue;
        }
        rt[idx]--;
        time++;
        timeline[len] = time;
        process[len] = idx+1;
        len++;
        if (rt[idx] == 0)
        {
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            finished[idx] = 1;
            completed++;
        }
    }
}

```

O/R

Enter number of processes: 5

Enter Arrival Time, Burst Time, Priority for P1: 0 3 5

Enter Arrival Time, Burst Time, Priority for P2: 2 2 3

Enter Arrival Time, Burst Time, Priority for P3: 3 5 2

Enter Arrival Time, Burst Time, Priority for P4: 4 4 4

Enter Arrival Time, Burst Time, Priority for P5: 5 1 1

-- Non-Preemptive Priority Scheduling --

PID	AT	BT	PR	CT	TAT	WT
1	0	3	5	3	3	0
2	2	2	3	11	9	7
3	3	5	2	8	5	0
4	4	4	4	15	11	7
5	5	1	1	9	4	8

Gantt chart:

| P1 | P3 | P5 | P2 | P4 |
0 3 8 9 11 15

Average Turnaround Time = 6.4

Average waiting Time = 3.4

-- Preemptive Priority Scheduling --

PID	AT	BT	PR	CT	TAT	WT
1	0	3	5	15	15	12
2	2	2	3	10	8	6
3	3	5	2	9	6	1
4	4	4	4	14	10	6
5	5	1	1	6	1	0

Gantt chart:

P1	P2	P2	P3	P3	P3	P3	P3	P2	P1	P1	P1	P1	P1	P1	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Average Turn Around Time = 8

Average waiting Time = 5

12/13/26

Lab 4

EDGE

Round Robin (Experiment with different quantum sizes for RR algorithm)

#include <stdio.h>

int main()

{ int n, tq;

int at[100], bt[100], rt[100];

int ct[100], tat[100], wt[100];

int queue[100], front = 0, rear = 0;

int waited[100] = {0};

int time = 0, completed = 0;

int gantt_p[100], gantt_t[100];

int g = 0;

float avg_wt = 0, avg_tat = 0;

printf("Enter no of processes:");

scanf("%d", &n);

printf("Enter Arrival Time: \n");

for(int i=0; i<n; i++)

{ printf("P%d: ", i+1);

scanf("%d", &at[i]);

}

printf("Enter Burst Time: \n");

for(int i=0; i<n; i++)

{

printf("P%d: ", i+1);

scanf("%d", &bt[i]);

rt[i] = bt[i];

}

printf("Enter Time Quantum:");

scanf("%d", &tq);

queue[rear++] = 0;

waited[0] = 1;

```
for(int i=0; i<g; i++)
    printf("%d ", garr[i]);
printf("\n");
printf("\n Process | E | A | T | B | C | T | A | W | n ");
```

```

for (int i = 0; i < m; i++)
{
    bal[i] = c[i] - a[i];
    wt[i] = bal[i] - b[i];
    avg_wt += wt[i];
    avg_bal += bal[i];
    printf("P/d %d %d %d %d %d %d %d\n", i+1, a[i], b[i], c[i], bal[i], wt[i]);
}

```

```

printf("n Average Waiting Time = %.2f", avg_wt/n);
printf("n Average Turnaround Time = %.2f", avg_tat/n);
return 0;

```

7

O/P

Enter number of processes: 5

Enter Arrival Time:

P1: 0

P2: 1

P3: 2

P4: 3

P5: 4

Enter Burst Time:

P1: 5

P2: 3

P3: 1

P4: 2

P5: 3

Enter Time quantum: 2

Gantt chart:

| P1 | P2 | P3 | P1 | P4 | P5 | P2 | P1 | P5 |

0 2 4 5 7 9 11 12 13 14

P1	P2	P3	P1	P4	P5	P2	P1	P5
0	2	4	5	7	9	11	12	13

Process	AT	BT	CT	TAT	WT
P1	0	5	13	13	8
P2	1	3	12	11	8
P3	2	1	5	3	2
P4	3	2	9	6	4
P5	4	3	14	10	7

Average waiting Time = 5.80

Average Turnaround Time = 8.60

Time quantum: 4

Gantt chart:

| P1 | P2 | P3 | P4 | P5 | P1 |
0 4 7 8 10 13 14

Process	AT	BT	CT	TAT	WT
P1	0	5	14	14	9
P2	1	3	4	6	3
P3	2	1	8	6	5
P4	3	2	10	7	5
P5	4	3	13	9	6

Average Waiting Time = 5.60

Average Turnaround Time = 8.40

Q20
Q21

Conclusion: As time quantum increases, ^{avg} TAT and WT decreases.

24/3/26

Lab 5

write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories: system processes and user processes. System processes are to be given higher priority than user processes. use FCFS scheduling for the processes in each queue.

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#define MAX 100

typedef struct

```
{
    int id;
    int arrival;
    int burst;
    int remaining;
    int completion;
    int waiting;
    int turnaround;
    int type;
}
```

} Process;

typedef struct

```
{
    Process queue[MAX];
    int front, rear;
}
```

} Queue;

void initQueue (Queue *q)

```
{
    q->front = q->rear = -1;
}
```

}

int isEmpty (Queue *q)

```
{
    return q->front == -1;
}
```

}

```
void enqueue (queue *q, Process p)
{ if (q->rear == MAX-1) return;
  if (q->front == -1) q->front = 0;
  q->queue[++q->rear] = p;
}
```

```
Process dequeue (queue *q)
{ Process p = q->queue[q->front];
  if (q->front == q->rear)
    { q->front = q->rear = -1;
    }
  else {
    q->front++;
  }
  return p;
}
```

```
int compareArrival (const void *a, const void *b)
{ Process *p1 = (Process *)a;
  Process *p2 = (Process *)b;
  return p1->arrival - p2->arrival;
}
```

```
int main ()
{ int n;
  printf ("Enter number of processes: ");
  scanf ("%d", &n);
  Process plist[MAX];
  for (int i=0; i<n; i++)
    { printf ("Process %d\n", i);
      printf ("Enter arrival time: ");
      scanf ("%d", &plist[i].arrival);
      printf ("Enter burst time: ");
      scanf ("%d", &plist[i].burst);
    }
```

DIP

a) All System Processes

Enter no of processes: 3

Process 0

Enter arrival time: 0

Enter burst time: 2

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 0

Process 2

Enter arrival time: 3

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

ID	Type	AT	BT	CT	WT	TAT
0	System	0	2	2	0	2
1	System	2	3	5	0	3
2	System	3	5	10	2	7

Average waiting Time: 0.67

Average Turnaround Time: 4.00

Gantt chart

P0	P0	P1	P1	P1	P2	P2	P2	P2	P2	P2
0	1	2	3	4	5	6	7	8	9	10

b) All user process

Enter no of processes: 3

Process 0

Enter arrival time: 0

Enter burst time: 2

Enter type: 1

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type: 1

Process 2

Enter arrival time: 3

Enter burst time: 5

Enter type: 1

ID	Type	AT	BT	CT	WT	TAT
0	user	0	2	2	0	2
1	user	2	3	5	0	3
2	user	3	5	10	2	7

Average waiting Time: 0.67

Average Turnaround Time: 4.00

Gantt chart:

	P0	P0	P1	P1	P1	P2	P2	P2	P2	P2
0	1	2	3	4	5	6	7	8	9	10

(C) user/system

Enter no of processes: 3

Process 0

Enter arrival time: 5

Enter burst time: 2

Enter type (0 = System, 1 = user): 0

Process 1

Enter arrival time: 0

Enter burst time: 4

Enter type: 1

Process 2

Enter arrival time: 1

Enter burst time: 3

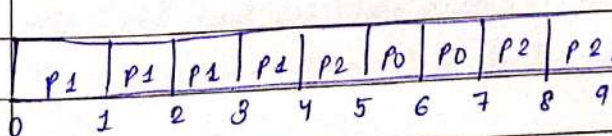
Enter type: 1

ID	Type	AT	BT	CT	WT	TAT
1	user	0	4	4	0	4
2	user	1	3	9	5	8
0	system	5	2	7	0	2

Average waiting Time: 1.67s

Average Turnaround Time: 4.67s

Gantt chart



difference between FCFS and Multilevel

FCFS

- Processes are scheduled in order they arrive
- Equal treatment for all processes
- Non-preemptive once a process starts, it runs until completion
- very simple

Multilevel

- Processes are divided into multiple queues based on type/priority
- Higher priority queues always get preference
- can be preemptive - lower priority processes maybe interrupted if higher priority process arrives
- complex

31/3/26

Lab 6

write a C program to simulate Real-Time CN scheduling algorithm

- Rate-Monotonic
- Earliest-deadline First
- Proportional scheduling

a) #include <stdio.h>

#include <math.h>

typedef struct T

int id, period, exec_time, remaining;

int waiting-time, turnaround_time, last-finish, completion_time;

} Task;

int gcd(int a, int b)

{ return (b == 0) ? a : gcd(b, a % b); }

}

int lcm(int a, int b)

{ return (a * b) / gcd(a, b); }

}

int compute_hypersperiod(Task tasks[], int n)

{ int h = tasks[0].period;

for (int i = 1; i < n; i++)

h = lcm(h, tasks[i].period);

return h;

}

double utilization_bound(int n)

{ return n * (pow(2.0, 1.0/n) - 1); }

}

void print_gantt_chart(int schedule[], int length)

{ printf("\n Rate Monotonic Gantt chart\n"); }

```
U += (double) tasks[i].exec_time / tasks[i].period;
```

```
}
```

```
double bound = utilization_bound(n);
```

```
printf("\n Total utilization U = %.3f", U);
```

```
printf("\n Utilization bound for  $\tau$  tasks = %.3f", n, bound);
```

```
if (U > bound)
```

```
{ printf("Task set is NOT guaranteed schedulable under RMS.\n");
  return 0;
```

```
}
```

```
printf("Task set is GUARANTEED schedulable under RMS.\n");
```

```
int hyperperiod = compute_hyperperiod(tasks, n);
```

```
int schedule[hyperperiod];
```

```
for (int t = 0; t < hyperperiod; t++)
```

```
{ int chosen = -1;
```

```
for (int i = 0; i < n; i++)
```

```
{ if (t % tasks[i].period == 0)
```

```
{ tasks[i].remaining = tasks[i].exec_time;
```

```
tasks[i].waiting_time += (t - tasks[i].last_finish);
```

```
}
```

```
if (tasks[i].remaining > 0)
```

```
{ if (chosen == -1 || tasks[i].period < tasks[chosen].period)
```

```
chosen = i;
```

```
}
```

```
}
```

```
if (chosen != -1)
```

```
{ tasks[chosen].remaining --;
```

```
schedule[t] = tasks[chosen].id;
```

```
if (tasks[chosen].remaining == 0)
```

```
{ tasks[chosen].last_finish = t + 1;
```

```
tasks[chosen].completion_time = t + 1;
```

```
tasks[chosen].turnaround_time += (t + 1);
```

```

} else schedule[i] = -1;
}
print_gantt_chart(schedule, hyperperiod);
printf("\n ID | ECT | Period | CT | WT | TAT | n");
double avg_wait = 0, avg_turn = 0;
for(int i = 0; i < n; i++)
{
    printf("%d | %d | %d | %d | %d | %d | %d | n", tasks[i].id, tasks[i].exec_time,
    double tasks[i].period, tasks[i].completion_time, tasks[i].waiting_time,
    tasks[i].turnaround_time);
    avg_wait += tasks[i].waiting_time;
    avg_turn += tasks[i].turnaround_time;
}
printf("\n Average Waiting Time: %.2f", avg_wait/n);
printf("\n Average Turnaround Time: %.2f", avg_turn/n);
return 0;
}

```

Output

a) Enter number of tasks: 3

Enter Task 1 (Period ExecTime): 20 3

Enter Task 2 (Period ExecTime): 5 2

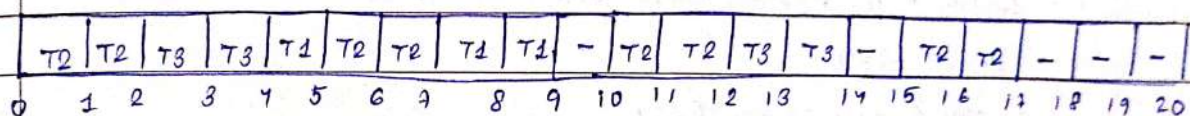
Enter Task 3 (Period ExecTime): 10 2

Total utilization $U = 0.750$

utilization Band for 3 tasks = 0.780

Task set is GUARANTEED schedulable under RMS

Rate Monotonic Gantt chart



ID	BT	Period	CT	WT	TAT
1	3	20	9	0	9
2	2	5	14	9	38
3	2	10	14	6	18

Average waiting Time: 5.00

Average Turnaround Time: 21.67

b) Enter number of tasks: 3

Enter Task 1 (Period ExecTime): 5 2

Enter Task 2 (Period ExecTime): 7 2

Enter Task 3 (Period ExecTime): 8 3

Total utilization $U = 1.061$

utilization Band for 3 tasks = 0.780

Task set is NOT guaranteed schedulable under RMS

(b) Earliest Deadline First

```
#include <stdio.h>
```

```
typedef struct
```

```
{
```

```
    int id, period, exec-time, deadline, remaining;
```

```
    int waiting-time, turnaround-time, last-finish, completion-time;
```

```
} Task;
```

```
void print_gantt_chart(int schedule[], int length)
```

```
{
```

```
    printf("\n Earliest Deadline First Gantt Chart \n");
```

```
    for(int i=0; i<length; i++) printf("----");
```

```
    printf("\n");
```

```
    for(int i=0; i<length; i++)
```

```
        { if (schedule[i] == -1) printf("|-");
```

```
          else printf("|T-%d", schedule[i]);
```

```
}
```

```
    printf("\n");
```

```
    for(int i=0; i<length; i++) printf("----");
```

```
    printf("\n");
```

```
    for(int i=0; i<length; i++) printf("T-%d", i);
```

```
    printf("\n");
```

```
}
```

```
int main()
```

```
{ int n;
```

```
    printf("Enter number of tasks:");
```

```
    scanf("%d", &n);
```

```
    Task tasks[n];
```

```
    double U = 0.0;
```

```
    for(int i=0; i<n; i++)
```

```

    printf("Enter Task /d (Period Deadline ExecTime) :", i+1);
    scanf("%d %d %d", &tasks[i].period, &tasks[i].deadline, &tasks[i].exec_time);
    tasks[i].id = i+1;
    tasks[i].remaining = 0;
    tasks[i].waiting_time = 0;
    tasks[i].turnaround_time = 0;
    tasks[i].last_finish = 0;
    tasks[i].completion_time = 0;
    U += (double) tasks[i].exec_time / tasks[i].period;

```

```

}
printf("In Total Utilization U = %.3f\n", U);
if (U > 1.0)
{
    printf("Task set is NOT schedulable under EDF.\n");
    return 0;
}

```

```

}
printf("Task set is SCHEDULABLE under EDF.\n");
int horizon = 20;
int schedule[horizon];
int deadlines[n];
for (int i = 0; i < n; i++)
    deadlines[i] = tasks[i].deadline;
for (int t = 0; t < horizon; t++)
{
    int chosen = -1;
    for (int i = 0; i < n; i++)
    {
        if (t / tasks[i].period == 0)
        {
            tasks[i].remaining = tasks[i].exec_time;
            deadlines[i] = t + tasks[i].deadline;
            tasks[i].waiting_time += (t - tasks[i].last_finish);
        }
        if (tasks[i].remaining > 0)
        {
            if (chosen == -1 || deadlines[i] < deadlines[chosen])
                chosen = i;
        }
    }
}

```

O/P

Enter Task 1 (Period Deadline ExecTime): 20

1

3

Enter Task 2 (Period Deadline ExecTime): 5

4

2

Enter Task 3 (Period Deadline ExecTime): 10

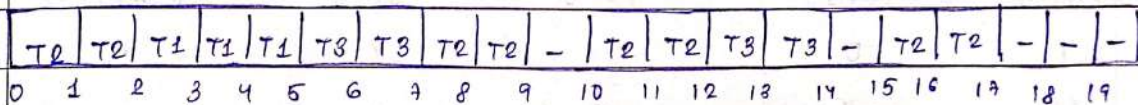
8

2

Total utilization $U = 0.750$

Task set is SCHEDULABLE under EDF.

Earliest Deadline First Gantt chart



ID	BT	Period	Deadline	CT	WT	TAT
1	3	20	7	5	0	5
2	2	5	4	17	7	40
3	2	10	8	14	3	21

Average waiting Time: 3.33

Average Turnaround Time: 22.00

211

Lab 2

write a c program to simulate:

- Producer - consumer problem using semaphores.
- Dining - philosopher's problem

```
(a) #include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 3
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void produce_item(int item)
{
    sem_wait(&empty);
    pthread_mutex_lock(&mutex);
    buffer[in] = item;
    printf("Produced: %d at buffer[%d]\n", item, in);
    in = (in + 1) % BUFFER_SIZE;
    pthread_mutex_unlock(&mutex);
    sem_post(&full);
}

void consume_item()
{
    int item;
    sem_wait(&full);
    pthread_mutex_lock(&mutex);
    item = buffer[out];
    printf("Consumed: %d from buffer[%d]\n", item, out);
```

```
out = (out + 1) % BUFFER_SIZE;  
pthread_mutex_unlock(&mutex);  
sem_post(&empty);
```

}

```
int main()
```

```
{ sem_init(&empty, 0, BUFFER_SIZE);  
  sem_init(&full, 0, 0);  
  pthread_mutex_init(&mutex, NULL);  
  produce_item(0);  
  sleep(1);  
  produce_item(1);  
  sleep(1);  
  produce_item(2);  
  sleep(1);  
  consume_item();  
  sleep(1);  
  produce_item(3);  
  sleep(1);
```

```
// cleanup
```

```
sem_destroy(&empty);  
sem_destroy(&full);  
pthread_mutex_destroy(&mutex);  
return 0;
```

}

O/P

Order: P P P P

Producers

Produced: 0 at buffer[0]

Produced: 1 at buffer[1]

Produced: 2 at buffer[0]

Consumed: 0 from buffer[0]

Produced: 3 at buffer[0]

Trace

Buffer size = 3

in = 0

out = 0

empty = 3

mux = 1

full = 0

① mp = 0

empty = 3 - 2

mux 1 → 0

buffer[0] = 0

Produced: 0 at buffer[0]

in = (0 + 1) / 3 = 1

mux 0 → 1

full 0 → 1

② mp = 1

empty 2 → 1

mux 1 → 0

buffer[1] = 1

Produced: 1 at buffer[1]

in = (1 + 1) / 3 = 2

mux 0 → 1

full 1 → 2

③ mp = 2

empty 1 → 0

mux 1 → 0

buffer[2] = 2

in = (2 + 1) / 3 = 0

mux 0 → 1

full 2 → 3

④

full 3 → 2

mux 1 → 0

item = buffer[0]

Consumed: 0 from buffer[0]

out = (0 + 1) / 3 = 1

mux 0 → 1, empty 0 → 1

(b)

Dining Philosopher

#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define N 5

sem_t chopstick[N];

sem_t room;

void *philosopher (void *num)

{ int i = *(int *) num;

while (1)

{ printf("Philosopher %d is thinking.\n", i);
sleep(1);

sem_wait(&room);

sem_wait(&chopstick[i]);

printf("Philosopher %d picked up left chopstick.\n", i);

sem_wait(&chopstick[(i+1)%N]);

printf("Philosopher %d picked up right chopstick.\n", i, (i+1)%N);

printf("Philosopher %d is eating.\n", i);

sleep(2);

sem_post(&chopstick[i]);

sem_post(&chopstick[(i+1)%N]);

printf("Philosopher %d put down chopsticks.\n", i, (i+1)%N);

sem_post(&room);

O/P

Philosopher 1 is thinking

Philosopher 0 is thinking

Philosopher 4 is thinking

Philosopher 2 is thinking

Philosopher 3 is thinking

Philosopher 1 picked up left chopstick 1

Philosopher 2 picked up left chopstick 2

Philosopher 2 picked up right chopstick 3.

Philosopher 2 is eating.

28/4/20

Lab 8 ^{Bankers} (Banking Safety Problem)

```
#include <stdio.h>

int main()
{
    int n, m;
    int i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int Allocation[m][n], Max[m][n], Need[m][n];
    int Available[m];
    printf("Enter Allocation matrix: \n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<m; j++)
        {
            scanf("%d", &Allocation[i][j]);
        }
    }
    printf("Enter Maximum Matrix: \n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<m; j++)
        {
            scanf("%d", &Max[i][j]);
        }
    }
    printf("Enter Available Resources: \n");
    for(j=0; j<m; j++)
    {
        scanf("%d", &Available[j]);
    }
    printf("Need matrix");
    for(i=0; i<n; i++)
    {

```

O/P

Enter number of processes: 5

Enter number of resources: 4

Enter Allocation Matrix:

0 0 1 2

1 0 0 0

1 3 5 4

0 6 3 2

0 0 1 4

Enter Maximum Matrix:

0 0 1 2

1 2 5 0

2 3 5 6

0 6 5 2

0 6 5 6

Enter Available Resources:

1 5 2 0

Need Matrix:

0 0 0 0

0 2 5 0

1 0 0 2

0 0 2 0

0 6 4 2

System is in a SAFE STATE.

Safe Sequence: P0 P2 P3 P4 P1

P2 Finish[2] = false

$$1008 \leq 1532$$

$$work = work + Allocation$$

$$3532 + 1354$$

$$= 2886$$

Finish[2] = true;

P3 Finish[3] = false

$$0020 \leq 2886$$

$$work = work + Allocation$$

$$= 2886 + 0632$$

$$= 214118$$

Finish[3] = true

P4 Finish[4] = false

$$0642 \leq 214118$$

$$work = 214118 + 0014$$

$$= 2141212$$

Finish[4] = true

P1 Finish[1] = false

$$0750 \leq 2141212$$

$$work = work + Allocation$$

$$= 2141212 + 1000$$

$$= 3141212$$

Finish[1] = true

ser
RA

Safe sequence = { P0, P2, P3, P4, P1 }

5/5/26

Lab-9Bankers Algorithm
(Resource Request)

```
int process;  
printf("\n Enter process number making request : ");  
scanf("%d", &process);  
int Request[m];  
int R  
printf("Enter request vector : ");  
for(j=0; j<m; j++)  
    scanf("%d", &Request[j]);  
}  
for(j=0; j<m; j++)  
    if(Request[j] > Need[process][j])  
        printf("\n Error: Request exceeds maximum need. \n");  
        return 0;  
}  
for(j=0; j<m; j++)  
    if(Request[j] > Available[j])  
        printf("\n Resources not available. Process must wait. \n");  
        return 0;  
}  
for(j=0; j<m; j++)  
    Available[j] -= Request[j];  
    Allocation[process][j] += Request[j];  
    Need[process][j] -= Request[j];  
}
```

O/P

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

Need Matrix:

7 4 3

1 0 2

6 0 0

0 1 1

4 3 1

Enter process number making request: 4

Enter request vector:

3 3 0

Request cannot be granted (unsafe state).

Enter process number making request: 1

Enter request vector:

1 0 2

(ii) Write a C program to simulate Deadlock Detection

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, m;
```

```
  int i, j, k;
```

```
  printf("Enter number of processes:");
```

```
  scanf("%d", &n);
```

```
  printf("Enter number of resource types:");
```

```
  scanf("%d", &m);
```

```
  int Allocation[m][m], Request[m][m], Available[m];
```

```
  printf("Enter Allocation Matrix: \n");
```

```
  for(i=0; i<n; i++)
```

```
  { for(j=0; j<m; j++)
```

```
    scanf("%d", &Allocation[i][j]);
```

```
  }
```

```
}
```

```
  printf("Enter Request Matrix: \n");
```

```
  for(i=0; i<n; i++)
```

```
  { for(j=0; j<m; j++)
```

```
    scanf("%d", &Request[i][j]);
```

```
  }
```

```
}
```

```
  printf("Enter Available Resources: \n");
```

```
  for(j=0; j<m; j++)
```

```
  { scanf("%d", &Available[j]);
```

```
  }
```

```
  int Finish[m], Work[m], Sequence[m];
```

```
  for(i=0; i<n; i++)
```

```
    Finish[i]=0;
```

```
  for(j=0; j<m; j++)
```

```
    Work[j] = Available[j];
```

```

printf("No deadlocked processes.");
for (i=0; i < m; i++)
{
    if (finish[i] == 0)
        printf("P%d", i);
}
printf("\n");
return 0;

```

O/P

Enter number of processes: 4

Enter number of resource types: 3

Enter Allocation Matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request Matrix:

0 0 1

1 0 2

0 0 0

3 3 0

Enter available Resources:

0 0 0

Deadlock detected!

Processes that can finish partial sequence: P2 P0 P1

Deadlocked Processes: P3

Enter number of processes: 5

Enter number of resource types: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 0 2

Enter Available Resources:

0 0 0

No Deadlock detected!

Execution Sequence: P0 P2 P3 P4 P1

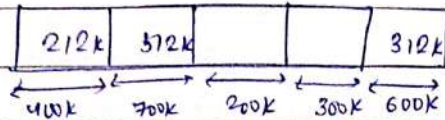
See
R.H.

12/6/22

Lab 10

- (Q) Given a free memory partition 400K, 700K, 200K, 300K and 600K in order. How would each of first fit, best fit and worst fit algorithms work to place processes of 212K, 512K, 312K, 526K (in order)
 P_1 P_2 P_3 P_4

First Fit (in order)



$P_1 - 212K$

$P_2 - 512K$

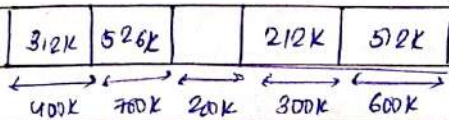
$P_3 - 312K$

$P_4 - 526K$

526K cannot allocated with memory

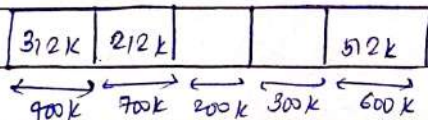
Best Fit (Nearest)

(see nearest val)



Worst Fit (Biggest)

526K cannot be allocated



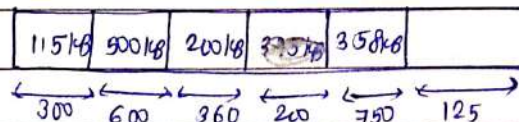
Memory partition

- (Q) 300KB, 600KB, 350KB, 200KB, 750KB (in order)

P1

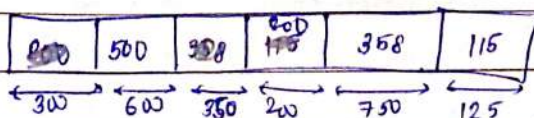
125KB

Process size 115KB, 500KB, 350KB, 200, 375



First Fit
 Process with size 375KB cannot be allocated with memory

Best Fit



Process with size 375KB cannot be allocated with memory

Write a C program to simulate the following contiguous memory allocation techniques:

- worst-fit
- Best-fit
- First-fit

```
#include <stdio.h>
```

```
void firstfit(int blocksize[], int blocks, int processsize[], int processes)
```

```
{ int allocation[processes];
```

```
for (int i=0; i < processes; i++)
```

```
allocation[i] = -1;
```

```
for (int i=0; i < processes; i++)
```

```
{ for (int j=0; j < blocks; j++)
```

```
{ if (blocksize[j] >= processsize[i])
```

```
{ allocation[i] = j;
```

```
blocksize[j] = 0;
```

```
break;
```

```
}
```

```
printf("In FIRST FIT ALLOCATION \n");
```

```
printf("Process No | Process size | Block No | \n");
```

```
for (int i=0; i < processes; i++)
```

```
{ printf("%d | %d | %d | \n", i+1, processsize[i],
```

```
if (allocation[i] != -1)
```

```
printf("%d \n", allocation[i]+1);
```

```
else
```

```
printf("Not Allocated \n");
```

```
}
```

```
void bestfit(int blocksize[], int blocks, int processsize[], int processes)
```

```
{ int allocation[processes];
```

```
for (int i=0; i < processes; i++)
```

```
allocation[i] = -1;
```

```
for (int i=0; i < processes; i++)
```

firstfit (block 1, blocks, percentage, processes);
 bestfit (block 2, blocks, percentage, processes);
 worstfit (block 3, blocks, percentage, processes);
 return 0;

J

o/p

Enter no of memory blocks: 6
 Enter size of memory blocks:

300

600

350

200

750

125

Enter no of processes: 5

Enter size of process:

115

500

358

200

375

FIRST FIT ALLOCATION

Process NO	Process size	Block NO
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not allocated

BEST FIT ALLOCATION

Process No	Process Size	Block No
1	115	6
2	500	2
3	358	5
4	200	4
5	375	Not Allocated

WORST FIT ALLOCATION

Process No	Process Size	Block No
1	115	5
2	500	2
3	358	Not Allocated
4	200	3
5	375	Not Allocated

25
28

26/5/26

Lab-11

Write a C program to simulate page replacement algorithm

- a) FIFO
- b) LRU
- c) Optimal

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int search(int key, int frame[], int n)
```

```
{
```

```
    for(int i=0; i<n; i++)
```

```
    { if(frame[i] == key) return i;
```

```
    }
```

```
    return -1;
```

```
}
```

```
int fifo(int pages[], int n, int frames)
```

```
{ int frame[frames], count=0, index=0;
```

```
    for(int i=0; i<frames; i++)
```

```
        frame[i] = -1;
```

```
    for(int i=0; i<n; i++)
```

```
    { if(search(pages[i], frame, frames) == -1)
```

```
        frame[index] = pages[i];
```

```
        index = (index+1) % frames;
```

```
        count++;
```

```
    }
```

```
    return count;
```

```
}
```

```
int lru(int pages[], int n, int frames)
```

```
{
```

```
    int frame[frames], count=0, time[frame];
```

```
    for(int i=0; i<frames; i++)
```

```

{ frames[i] = -1;
  time[i] = -1;

```

```

}
for (int i = 0; i < n; i++)

```

```

{
  int pos = search(pages[i], frame, frames);
  if (pos == -1)

```

```

{
  int min = 0;
  for (int j = 1; j < frames; j++)
    if (time[j] < time[min]) min = j;

```

```

}
frame[min] = pages[i];
time[min] = i;
count++;

```

```

} else {
  time[pos] = i;

```

```

}
return count;

```

```

}

int optimal (int pages[], int n, int frames)

```

```

{
  int frame[frames], count = 0;
  for (i = 0; i < n; i++)
    if (replace == -1, farthest = i;

```

```

    for (int j = 0; j < frames; j++)
      if (K == 0)

```

```

        replace = j;

```

printTable (n, frame, numframes);

return 0;

}

O/P

Enter no of pages: 20

Enter ref string : 70 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Enter how many framesizes: 2

Enter size of frame1: 3

Enter size of frame2: 4

algo	3 frames	4 frames
FIFO	15	10
LRU	12	8
Optimal	9	8

Total pages = 20

see
SS